

Nawrin Zawna A N

Linux Task 03 — Process Management, System Monitoring & Basic Shell Scripting

Part A: Process Monitoring

Commands Used:

```
$ ps                # Shows processes running in current terminal
$ ps aux            # Shows all running processes for all users
$ top               # Real-time dynamic view of running processes
$ sudo apt install htop # Install htop if not available
$ htop              # Interactive, color-coded process viewer
```

Q1. What is a Process?

A process is a running instance of a program. When you execute a command or open an application in Linux, the operating system creates a process for it. Each process has its own memory space, resources, and state. Processes can be in running, sleeping, stopped, or zombie states.

Q2. What is a PID?

PID stands for Process ID. It is a unique numerical identifier assigned by the Linux kernel to every process when it starts. The PID is used to track, manage, and control processes — for example, to send signals or terminate them using the kill command.

Q3. Which process consumes the most CPU?

In the 'top' command, processes are sorted by CPU usage by default. Typically, system processes like 'Xorg', 'compiz', or a running browser/application appear at the top. The process at the very top of the list (column %CPU) is consuming the most CPU. Example: 'top' or 'htop' — press P to sort by CPU.

Q4. Which process consumes the most Memory?

In 'top', press M to sort by memory usage (%MEM column). Typically, browsers, IDEs, or desktop environments consume the most RAM. In 'ps aux', sort by the %MEM column: `ps aux --sort=-%mem | head -5`

Part B: Process Management

Steps Performed:

```
# Step 1: Open terminal and run a background sleep process
$ sleep 300 &

# Step 2: Find the process PID
$ ps aux | grep sleep
# Example output:
# student  1234  0.0  0.0   7228   820 pts/0    S   10:05   0:00 sleep 300

# Step 3: Terminate using normal kill
$ kill 1234

# Step 4: Force kill if process doesn't stop
$ kill -9 1234
```

Documentation:

PID Found: 1234 (example — varies on each run)

Command Used: `ps aux | grep sleep → kill 1234 → kill -9 1234`

Result: 'kill 1234' sends SIGTERM (signal 15) — a graceful termination request. The process is given a chance to clean up. 'kill -9 1234' sends SIGKILL (signal 9) — forces immediate termination with no

cleanup. Use -9 when a process refuses to stop with a normal kill.

Part C: System Monitoring

Commands Used:

```
$ free -h          # Shows RAM usage in human-readable format
$ df -h           # Shows disk space usage
$ uptime          # Shows how long the system has been running
$ uname -a        # Shows full kernel and system information
```

System Summary Report (Sample Output):

```
===== System Summary Report =====
```

```
Total RAM       : 8.0 GB
Available RAM    : 3.2 GB
Used RAM         : 4.8 GB
```

```
Disk Usage      : /dev/sda1  50G  used 20G  free 30G  (40% used)
```

```
System Uptime   : up 2 hours, 35 minutes, 1 user, load average: 0.45, 0.38, 0.30
```

```
Kernel Version  : Linux kali 6.1.0-kali9-amd64 #1 SMP PREEMPT_DYNAMIC
                  Debian 6.1.27-1kali1 (2023-05-12) x86_64 GNU/Linux
```

```
=====
```

Part D: Service Monitoring

Commands Used:

```
$ systemctl status ssh          # Check SSH service status
$ systemctl status NetworkManager # Check Network Manager status
$ systemctl status apache2      # Alternative if above unavailable
```

Q1. What is a Service?

A service (also called a daemon) is a background process that runs continuously and provides specific functionality to the operating system or users. Services start automatically at boot and run without user interaction. Examples: SSH (remote access), NetworkManager (internet), Apache (web server), cron (task scheduling).

Q2. Why are Services Important?

Services are the backbone of a Linux system. They handle critical functions like network connectivity, security (SSH, firewall), logging, and application hosting. Without key services, the system cannot perform its intended role. In cybersecurity, monitoring services helps detect unauthorized or suspicious processes running on a system.

Q3. How can a Stopped Service Affect a System?

If a critical service stops unexpectedly, it can cause system failures, data loss, or security vulnerabilities. For example: SSH stopped = no remote access to the server; NetworkManager stopped = loss of internet connectivity; firewall stopped = system exposed to attacks; logging service stopped = no audit trail for security incidents.

Part E: Shell Scripting — system_report.sh

Script: system_report.sh

```
#!/bin/bash
```

```
# system_report.sh - System Information Report Script
# Author: Nawrin Zawna A N
```

```
echo "=====
echo "          System Information Report          "
echo "=====
echo ""
echo "User           : $(whoami)"
echo "Hostname        : $(hostname)"
echo "Date & Time      : $(date '+%d-%m-%Y %H:%M:%S')"
echo "Current Dir      : $(pwd)"
echo ""
echo "----- Memory Usage -----"
free -h
echo ""
echo "----- Disk Usage -----"
df -h
echo ""
echo "----- System Uptime -----"
uptime
echo ""
echo "=====
echo "          End of Report          "
echo "=====
```

How to Execute:

```
$ chmod +x system_report.sh    # Give execute permission
$ ./system_report.sh           # Run the script
```

Sample Output:

```
=====
          System Information Report
=====

User           : student
Hostname        : kali
Date & Time     : 04-06-2025 10:30:45
Current Dir     : /home/student

----- Memory Usage -----
              total    used    free   shared  buff/cache  available
Mem:          7.7G    3.2G    1.5G    312M        3.0G        4.0G
Swap:         975M     0B    975M

----- Disk Usage -----
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   20G   30G  40% /

----- System Uptime -----
10:30:45 up 2:35,  1 user,  load average: 0.45, 0.38, 0.30

=====
          End of Report
=====
```

Part F: Security Monitoring Challenge

Command: netstat

Purpose:

Displays active network connections, routing tables, interface statistics, and listening ports. It shows which ports are open and which processes are communicating over the network.

Example Output:

```
$ netstat -tulnp
Proto Local Address   Foreign Address  State       PID/Program
tcp   0.0.0.0:22      0.0.0.0:*        LISTEN      1234/sshd
tcp   127.0.0.1:3306  0.0.0.0:*        LISTEN      5678/mysqld
```

Security Use Case:

Used by SOC analysts to detect unauthorized open ports, suspicious outbound connections, or potential backdoors/malware communicating with external servers.

Command: ss

Purpose:

A faster, modern replacement for netstat. Displays socket statistics — shows established connections, listening ports, and network socket details.

Example Output:

```
$ ss -tulnp
Netid  State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
tcp    LISTEN  0        128     0.0.0.0:22          0.0.0.0:*
```

Security Use Case:

Used to quickly identify listening services and active connections. Faster than netstat and preferred in modern Linux systems for network forensics.

Command: who

Purpose:

Displays information about users currently logged into the system — their username, terminal, login time, and IP address.

Example Output:

```
$ who
student pts/0  2025-06-04 10:05 (192.168.1.10)
root    pts/1  2025-06-04 10:15 (192.168.1.20)
```

Security Use Case:

Used to detect unauthorized logins or unexpected root access. If an unknown IP is listed, it may indicate a security breach.

Command: w

Purpose:

An extended version of 'who' — shows logged-in users AND what they are currently doing, including CPU usage and idle time.

Example Output:

```
$ w
USER      TTY      FROM          LOGIN@      IDLE   WHAT
```

```
student pts/0 192.168.1.10 10:05 0.00s w
root pts/1 192.168.1.20 10:15 5:00 bash
```

Security Use Case:

Used to monitor active user sessions. Analysts check this to detect insider threats or attackers running suspicious commands on a compromised system.

Command: last

Purpose:

Shows a history of all user login and logout events read from `/var/log/wtmp`. Displays username, terminal, IP, and login/logout times.

Example Output:

```
$ last
student pts/0 192.168.1.10 Wed Jun 4 10:05 still logged in
root pts/1 192.168.1.20 Wed Jun 4 09:00 - 09:45 (00:45)
reboot system boot Wed Jun 4 08:55
```

Security Use Case:

Used in incident response to trace when accounts were accessed and from where. Unusual login times or unknown IPs are red flags for security investigations.

Part G: Mini SOC Activity

Scenario: I am a Security Analyst and the system is running slowly.

1. How would I identify resource-heavy processes?

I would start by running `top` or `htop` to get a real-time view of all running processes sorted by CPU and memory usage. The processes at the top of the list are consuming the most resources. I would press `P` in `top` to sort by CPU, and `M` to sort by memory. I would also run `ps aux --sort=-%cpu | head -10` to list the top 10 CPU-consuming processes and `ps aux --sort=-%mem | head -10` for memory. Additionally, `free -h` helps check if RAM is exhausted, and `df -h` checks if disk space is full — both can cause slowness.

2. How would I determine if a process is suspicious?

After identifying resource-heavy processes, I would examine them carefully. A process is suspicious if: it has an unusual or random name (e.g., `xmrig`, `kworker2` misspelled); it is run by root but was not started by an administrator; it is communicating with unknown external IPs (checked using `netstat -tulnp` or `ss -tulnp`); it has no clear parent process (PPID of 1 unexpectedly); it was started at an unusual time (checked with `ps aux` timestamp or `last` command); or the process path is in `/tmp` or `/var/tmp` instead of `/usr/bin`. I would cross-check the process name against known legitimate system processes.

3. What information would I collect before terminating a process?

Before terminating any process, I would collect: PID (Process ID) using `ps aux | grep '`; the full command and path using `ls -la /proc//exe`; the user who owns the process; network connections associated with it using `ss -tulnp | grep '`; open files using `ls -l -p '`; parent process ID (PPID) to trace its origin; and the start time of the process. I would document all this information as evidence before killing the process, because once terminated, forensic data may be lost. In a real SOC environment, I would also take a memory dump before killing a suspicious process for further malware analysis.